AI-103 Azure AI App and Agent Developer — Cram Guide

Microsoft Azure Certification

Version: v1 | Beta April 2026 / Live June 2026 | 2026

■ Exam at a Glance

Exam Code	AI-103
Certification	Azure AI App and Agent Developer Associate
Level	Associate
Questions	40–60
Duration	120 minutes
Passing Score	700 / 1000
Cost	~$165 USD (beta: ~$99)
Prerequisite	None formally — Python/C# experience strongly recommended
Predecessor	AI-102 (retires June 30, 2026)
Certification Expiry	Annual — free renewal via Microsoft Learn assessment

****AI-103 is Foundry-first.** If you studied AI-102, throw out the Cognitive Services approach. AI-103 is built around Microsoft Foundry (formerly Azure AI Studio) and agentic AI development.**

■ Domain Weightings

Domain	Weight	Key Focus
Plan and Manage Azure AI Solutions	25–30%	Foundry services, access control, monitoring, responsible AI
Implement Generative AI & Agentic Solutions	30–35%	RAG, agents, prompt engineering, multi-agent, Foundry SDK
Implement Computer Vision Solutions	10–15%	Vision models, GPT-4o vision, Custom Vision, video analysis
Implement Text Analysis Solutions	10–15%	Language models, Speech, Translator, CLU
Implement Information Extraction	10–15%	Document Intelligence, AI Search, vector search

****Domains 1 + 2 = 55–65% of the exam.** Master Foundry, RAG, and agents first.**

■ What is Microsoft Foundry?

Microsoft Foundry = Azure AI Foundry — unified platform for the entire AI lifecycle: model selection, prompt engineering, RAG, agent orchestration, evaluation, and deployment.

> Formerly Azure AI Studio. The name changed in 2025-2026. Same platform, new branding.

Foundry vs. Cognitive Services (AI-102 vs AI-103)

AI-102 (Legacy)	AI-103 (New)
Azure Cognitive Services	Azure AI Foundry
Pre-built REST APIs	Model catalog + custom prompts + agents
Individual API integration	End-to-end AI app design
No code / low code	Python/C# SDK, code-first
Limited customization	Full control: fine-tuning, RAG, agents

■ Foundry Model Catalog

Model Types — When to Use What

Model Type	Examples	Use When
LLM (Large Language Model)	GPT-4o, GPT-4o-mini, o1, o3, Llama 3, Phi-4	Complex reasoning, generation, RAG, agents
SLM (Small Language Model)	Phi-4-mini, Phi-3-mini	Low-latency, edge, constrained cost, simpler tasks
Multimodal	GPT-4o, Gemini 2.0, Llama Vision	Image + text + video understanding
Embedding	text-embedding-3-large, text-embedding-ada	RAG, semantic search, similarity
Speech	Whisper, Azure TTS, Speech-to-text	Voice interfaces
Vision	Azure AI Vision, DALL-E 3	Image generation, analysis

Model selection criteria:

- **Task complexity** — GPT-4o for complex reasoning; Phi for simple classification/summarization
- **Latency** — SLMs are faster and cheaper
- **Cost** — GPT-4o-mini is 10x cheaper than GPT-4o for many tasks
- **Multimodality** — Need vision? Use GPT-4o or Llama Vision
- **Context window** — GPT-4o: 128K tokens; Phi-4-mini: 4K-32K

Foundry Models vs. Azure OpenAI (What's the Difference?)

	Azure OpenAI Service	Foundry Models
What it is	Direct OpenAI models via Azure	Microsoft-hosted + partner models
Models	GPT-4, GPT-4o, Codex	GPT-4o, Llama, Phi, Mistral, Gemini, community
Deployment	Manual deployment in Azure OpenAI Studio	One-click deploy in Foundry
Management	Azure OpenAI resource	Foundry project
Same models?	Yes (same underlying GPT models)	Some overlap — GPT-4o available in both

****Use Foundry for new development.** Azure OpenAI is for when you specifically need OpenAI models with Azure's enterprise compliance.**

■ DOMAIN 1 — Plan and Manage Azure AI Solutions (25–30%)

Setting Up Foundry Projects

Project structure:

```
Foundry Subscription
■■■ Hub (organizational level: shared resources, quotas)
■■■ Project (isolated workspace)
■■■ Models (deployed models)
■■■ Agents (agent definitions)
■■■ Data (indexes, datasets)
■■■ Evaluations (quality assessments)
■■■ Deployments (endpoints)
```

Key differences:

- **Hub:** Organization-wide, shared model quotas, billing
- **Project:** Team-level isolation, individual budgets, RBAC

Choosing Deployment Options

Option	Use When
Pay-as-you-go (PAYG)	Variable usage, getting started, low volume
Provisioned Throughput Units (PTUs)	Predictable high volume, guaranteed capacity
Serverless API	Infrequent, bursty, don't want to manage capacity
Managed Online Endpoints	Production with SLA, auto-scale

PTUs vs. PAYG:

- PTU: Reserved capacity, lower latency at scale, monthly commitment
- PAYG: Pay per token, scales automatically, no commitment
- **PTU for production AI apps with consistent traffic**

Managing Access & Security

Authentication options:

Method	Use When
API Key	Simple, quick testing, not for production
Microsoft Entra ID (RBAC)	Production, team access control
Managed Identity	Azure-hosted apps (VM, App Service, AKS)
Keyless (Microsoft Entra ID only)	Best for production, no keys to rotate

Private networking:

- **Private endpoints** — Connect Foundry to your VNet, no public internet
- **AI Foundry virtual networks** — Managed VNet within Foundry
- Essential for enterprise / compliance scenarios

Content Safety:

- Azure AI Content Safety (formerly Azure Content Moderator)
- Built into Foundry: safety filters, jailbreak detection, protected material
- Custom safety evaluations — you define what's unsafe

Monitoring & Cost Management

What to monitor:

- Token usage per model/deployment
- Latency (time-to-first-token, total duration)
- Error rates (rate limit, auth, content filtered)
- Grounding quality (for RAG: how relevant are retrieved docs?)
- Model drift (is quality degrading over time?)

Cost optimization:

- Use GPT-4o-mini or Phi-4 for simpler tasks
- Set usage quotas per project/team
- Use PTUs for predictable high-volume
- Implement caching for repeated queries (semantic cache)

■ DOMAIN 2 — Implement Generative AI & Agentic Solutions (30–35%)

> ■ **This is the heaviest section. 30-35% of the exam.**

Prompt Engineering

Key techniques the exam tests:

Zero-shot prompting: Just describe the task in the prompt.

```
"Classify this review as Positive, Negative, or Neutral: [review]"
```

Few-shot prompting: Give examples before the actual question.

```
"Positive: I love this product
Negative: Terrible experience
Neutral: It works as described
Now classify: The battery lasts about 5 hours"
```

Chain-of-Thought (CoT): Ask the model to reason step by step.

```
"Think step by step: Should I invest in this company based on [data]?"
```

Structured outputs: Request JSON with specific schema.

```
"Return a JSON object with fields: sentiment, confidence, key_phrases"
```

System prompt: Defines the model's persona, rules, constraints.

```
"You are a helpful coding assistant that always explains your reasoning"
```

RAG — Retrieval Augmented Generation

RAG = Your data (grounding) + LLM reasoning.

Architecture components:

```
User Question
↓
[Embed Question] → text-embedding-3-large model
↓
[Vector Search] → Azure AI Search (index)
↓
[Top-K Chunks Retrieved]
↓
[Send to LLM with question + context]
↓
LLM generates answer grounded in your data
```

RAG patterns:

Pattern	How It Works	When to Use
Basic RAG	Retrieve → Augment → Generate	Simple Q&A;, single vector index
Parent Document RAG	Chunk documents, link child chunks to parent	Large docs, need full context
Hierarchical RAG	Summaries → detailed chunks	Very large corpuses
Query decomposition	Break complex question into sub-questions	Multi-hop reasoning
Hybrid search	Vector + keyword (BM25) search together	High accuracy needs

Azure AI Search (formerly Cognitive Search):

- Vector search: semantic similarity using embeddings
- BM25 keyword search: traditional full-text
- Hybrid: combines both (recommended)
- Semantic ranking: re-ranks results for relevance
- Knowledge store: persist enriched data to GCS/Table Storage

AI Agents — Foundry Agent Service

What is an agent? An AI system that:

Receives a goal
Plans steps to achieve it
Uses tools (function calling) to take actions
Iterates until goal is reached

Agent components:

```
Agent
■■■ Model (the LLM powering it)
■■■ Instructions (system prompt / role definition)
■■■ Tools (function calling: search, code execution, API calls)
■■■ Memory (conversation history, persistent context)
■■■ Orchestration (how it plans and executes steps)
```

Function calling (Tools):

- Define tools as functions in your code
- Model decides when to call a tool based on user intent
- Tools return data → model generates final response

```
# Example: Tool for searching a knowledge base
@tool
def search_knowledge_base(query: str) -> str:
"""Search the company knowledge base for the answer."""
results = ai_search.search(query, top=3)
return format_results(results)
```

Multi-agent orchestration:

```
User Request
↓
Orchestrator Agent (breaks down task)
↓
■■■ Agent A: Web Search
■■■ Agent B: Internal Database Query
■■■ Agent C: Document Retrieval
↓
Aggregator Agent (synthesizes responses)
↓
Final Response
```

Agent memory patterns:

- **Short-term:** Conversation history (within a session)
- **Long-term:** Persistent vector store for facts across sessions
- **Session summary:** Compress history to save context tokens

Evaluating AI Applications

Evaluation metrics:

Metric	What It Measures	When to Use
Groundedness	Did the model use only provided context?	RAG answers
Relevance	Are retrieved docs actually relevant to the query?	RAG retrieval
Coherence	Does the answer flow logically?	Generated text
Fluency	Is the text grammatically correct?	Generated text
Accuracy	Is the answer factually correct?	Factual Q&A;
Similarity	How close is the answer to a reference answer?	Comparing outputs

Azure AI evaluation:

- Built-in evaluators in Foundry
- Custom evaluators for domain-specific metrics
- Evaluations can be human-rated, model-rated, or statistical

---

■■ DOMAIN 3 — Implement Computer Vision Solutions (10–15%)

Azure AI Vision

Key capabilities:

- Image analysis: objects, tags, description, captions
- OCR: read text from images (receipts, documents)
- Spatial analysis: detect people and their movements in video
- Face detection: locate faces in images
- Video Indexer: analyze video for insights, transcripts, scene changes

GPT-4o Vision (multimodal):

- Accepts both images and text as input
- Describe images, answer questions about visuals
- Extract text from diagrams, charts, screenshots
- Use for: document understanding, visual Q&A;, image captioning

Custom Vision:

- **Image Classification:** Assign labels to entire images (is this a cat or dog?)

- **Object Detection:** Find and locate objects within images (bounding boxes)
- **Pre-trained models first:** Always check Azure AI Vision before building custom models
- Custom model training: upload labeled images → train → deploy as endpoint

Multimodal Workflow

```
Image Input
↓
Azure AI Vision or GPT-4o Vision (pre-process)
↓
[Extract: text? objects? scene description?]
↓
[Feed extracted info to LLM for reasoning] (RAG or agent)
↓
Final output
```

When to use Custom Vision vs. GPT-4o Vision:

- Custom Vision: Need to classify against your specific domain labels (parts, defects, species)
- GPT-4o Vision: General understanding, text extraction, visual reasoning

---

■ DOMAIN 4 — Implement Text Analysis Solutions (10–15%)

Language Models for Text Analysis

Azure AI Language:

- **Key Phrase Extraction:** Pull important topics from text
- **Entity Recognition (NER):** Identify people, organizations, locations, dates
- **Sentiment Analysis:** Positive/negative/neutral at document or sentence level
- **Language Detection:** Identify language from text
- **Text Summarization:** Abstractive or extractive summaries
- **Custom Language Models (CLU):** Train on your domain data for NER/intent classification

When to use pre-built vs. custom:

- Pre-built: Standard entity types, general sentiment
- Custom: Domain-specific entities (medical, legal, product SKUs), complex intent classification

Speech Services

Azure AI Speech:

Service	What It Does	Use Case
Speech-to-Text	Convert audio → text	Transcription, voice commands, meeting notes
Text-to-Speech (TTS)	Convert text → audio	Voice assistants, accessibility, audiobooks
Speech Translation	Real-time speech → text → another language	Live translation, call center
Custom Speech	Improve accuracy for domain-specific terms	Medical dictation, technical content
Voice profiles	Speaker identification and verification	Authentication, diarization

Speaker diarization: "Who said what" — useful for meeting transcription.

Pronunciation assessment: Evaluate spoken language (language learning apps).

Translator

Azure AI Translator:

- Neural Machine Translation (NMT) — more natural than statistical MT
- Document translation — translate whole documents while preserving layout
- Custom Translator — train domain-specific translation models
- **Dynamic Dictionary:** Override specific term translations

---

■ DOMAIN 5 — Implement Information Extraction (10–15%)

Azure AI Document Intelligence (formerly Form Recognizer)

What it does: Extract structured data from unstructured documents.

Models:

Model	Use For
Pre-built: Invoice	Extract line items, totals, dates from invoices
Pre-built: Receipt	Expense reports, loyalty cards
Pre-built: ID	Passports, driver licenses
Pre-built: W-2, 1098, etc.	US tax forms
Pre-built: Contract	Legal contract analysis
Pre-built: Document (general)	Layout, tables, key-value pairs from any doc
Custom model	Your specific documents with your layout

Layout API: Best for mixed-content documents — extracts text, tables, selection marks, structure without needing to train a model.

Training a custom model:

Upload labeled training documents (PDF, images)
Label fields: key-value pairs, tables, signatures
Train → get model ID
Analyze new documents with model ID
Deploy to Form Recognizer endpoint

Vector Search & Semantic Ranking

Vector search:

- Store embeddings (vector representations) of text chunks
- Query embedding → find most similar chunks
- Used in RAG for retrieving relevant context

Azure AI Search vector capabilities:

- `vectorizableTextQuery` — auto-generates embedding from text query
- Hybrid search — combines vector similarity + keyword (BM25)
- Semantic ranker — re-ranks results using Microsoft's semantic understanding
- Chunking strategies: fixed-size, by sentence, by paragraph

Index creation:

```
index = {
"name": "knowledge-base",
"fields": [
{"name": "id", "type": "Edm.String"},
{"name": "content", "type": "Edm.String", "searchable": True},
{"name": "content_vector", "type": "Collection(Edm.Single)",
"searchable": True, "vectorSearchDimensions": 1536,
"vectorSearchProfile": "default"}
]
}
```

Content Understanding

Azure AI Content Understanding:

- Process multimodal documents (PDF + images + tables)
- Extract structured data using custom models
- Pre-built templates for common document types
- Use when Document Intelligence isn't flexible enough

■■ Implementing RAG End-to-End

Step-by-step:

```
1. Ingest documents
→ Load (PDF, DOCX, TXT from Blob Storage / SharePoint)
→ Chunk (fixed size or semantic splitting)
→ Embed (text-embedding-3-large or ada)
→ Store in Azure AI Search index

2. Query the RAG pipeline
→ User question
→ Embed question
→ Vector search (top-K results)
→ Hybrid search if enabled
→ Retrieve context chunks
→ Send to LLM (GPT-4o) with system prompt + context

3. Evaluate
→ Run evaluators (groundedness, relevance, coherence)
→ Monitor quality over time
→ Retrain/re-index if quality drops
```

Orchestrating with Azure AI Foundry SDK:

```
from azure.ai.projects import AIProjectClient
from azure.identity import DefaultAzureCredential

client = AIProjectClient.from_connection_string(
connection_string="...",
credential=DefaultAzureCredential()
)

# Create agent with tools
agent = client.agents.create_agent(
model="gpt-4o",
instructions="You are a helpful assistant.",
tools=[search_tool, code_execution_tool]
)

# Send message
response = agent.send_message("What is the return policy?")
print(response.output)
```

■■ Responsible AI

What the exam expects you to know:

Content Safety in Foundry:

- **Safety filters:** Built-in filtering for jailbreak, protected material, violence
- **Groundedness:** Prevents hallucinations by forcing LLM to cite retrieved docs
- **Jailbreak detection:** Blocks prompt injection attacks
- **Custom evaluators:** Define domain-specific safety checks

AI Inspector (Foundry):

- Review traces of agent decisions
- Audit tool calls, context used, responses generated
- Safety evaluations: run red-team prompts against your agent

Key principles:

- **Fairness:** Don't introduce bias based on protected characteristics
- **Privacy:** Remove PII from training data (use DLP tools)
- **Transparency:** Users should know they're interacting with AI
- **Accountability:** Document model choices, data sources, limitations

---

■ Decision Trees

Which Model?

```
Need general-purpose reasoning, code, RAG?
■■■ YES → GPT-4o (complex) or GPT-4o-mini (simpler, cheaper)

Need very low latency / cheap?
■■■ YES → Phi-4-mini or Phi-3-mini

Need vision (image + text)?
■■■ YES → GPT-4o (multimodal) or Llama Vision

Need speech?
■■■ YES → Whisper (STT), Azure TTS

Need custom domain NER/intent?
■■■ YES → Custom Language Model (CLU)
```

RAG vs. Fine-tuning?

Scenario	Approach
Need model to answer from your documents	RAG (grounding)
Need consistent output format/instructions	System prompt + few-shot
Need model to learn domain terminology	Fine-tuning (expensive, for high-volume)
Need model to take actions	Agent with tools
Need both knowledge + actions	RAG + Agent

Document Extraction?

```
Need to extract from invoices, receipts, IDs?
■■■ YES → Pre-built Document Intelligence model

Need to extract from custom documents (your layout)?
■■■ YES → Custom Document Intelligence model

Need mixed content with tables and figures?
■■■ YES → Layout API or Content Understanding

Need to search extracted docs semantically?
■■■ YES → Azure AI Search (vector + hybrid)
```

■ Exam Day Tips

Foundry is the platform — AI-103 is code-first with the Foundry SDK, not portal clicks
RAG is everywhere — most scenarios involve grounding LLM responses in your own data
Agents = tools + memory — the exam tests your ability to define tools and manage context
Content Safety is mandatory — always mention grounding, safety filters, responsible AI
Pre-built first — use pre-built Document Intelligence before training custom models
Hybrid search > pure vector — when in doubt, use Azure AI Search with vector + BM25
Phi for simple tasks — don't use GPT-4o when Phi-4-mini will do
Custom Language Models (CLU) for domain NER, not GPT-4o
AI Foundry SDK — know how to create agents, add tools, send messages
Azure AI Content Safety — always integrated in Foundry, not optional

---

■ Official Resources

- AI-103 on Microsoft Learn
- Microsoft Foundry Documentation
- AI-103 Study Guide (Microsoft)
- Azure AI Foundry Agent Service

Study hard. Pass the exam. ■

Last updated: June 2026